

Optimization Algorithms



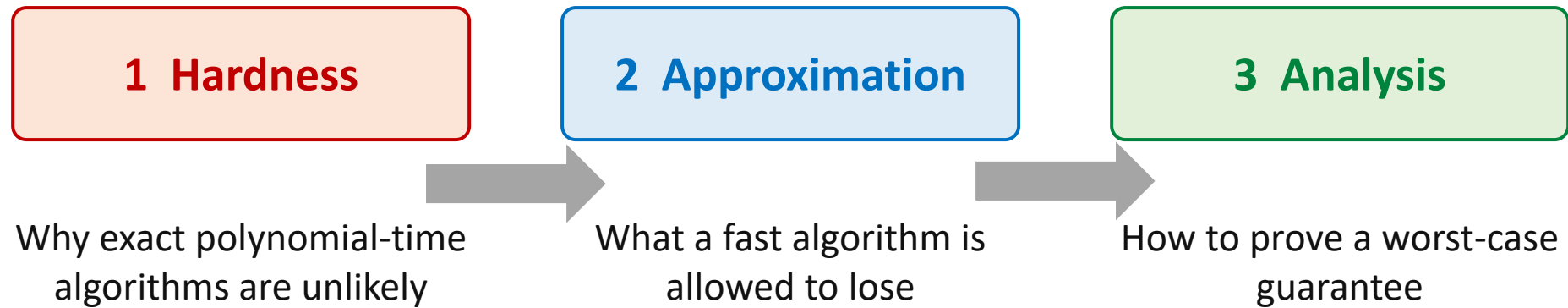
CS4234

Week 1

Introduction to
Approximation Algorithms

Diptarka Chakraborty

From hardness to useful guarantees



Two worked examples: knapsack first, then set cover.

Decision and optimization ask different questions

Decision

Input: graph G , vertices s and t , budget B

Is there an s – t path of length $\leq B$?

Optimization

Input: graph G , vertices s and t

What is the shortest s – t path length?

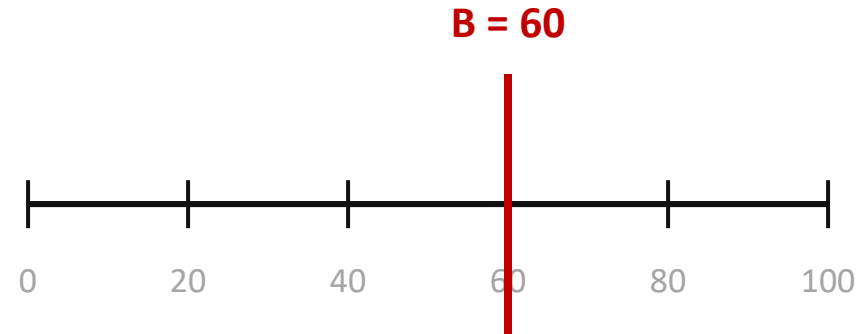
Decision returns YES/NO; optimization returns the best value or solution.

A threshold turns optimization into a decision query

Minimization problem

Given budget B , ask:

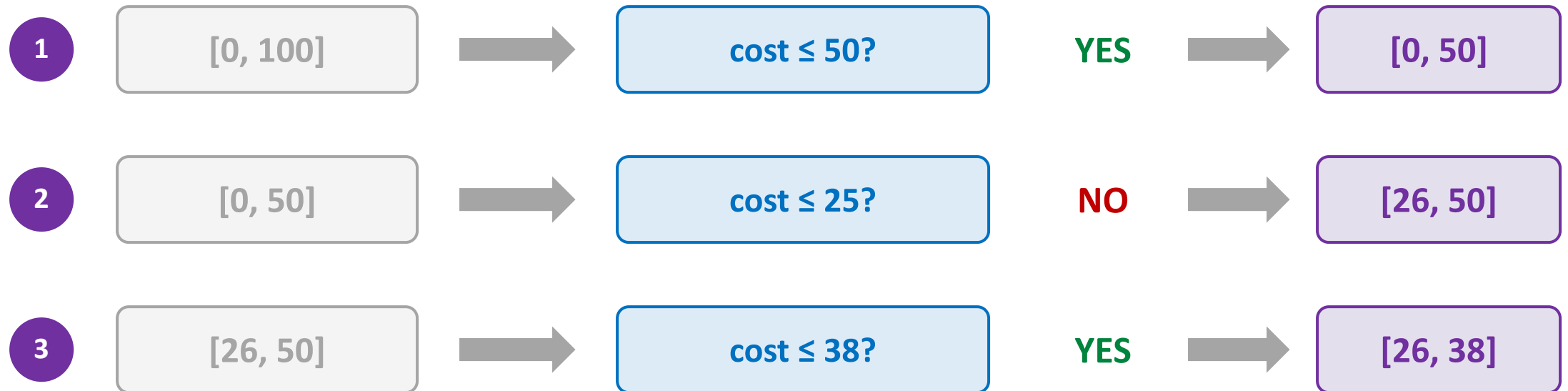
Does a feasible solution
with cost $\leq B$ exist?



YES if the optimum lies here

An optimizer immediately answers every threshold question.

Repeated decision queries can locate the optimum



For bounded integer objectives, binary search uses only logarithmically many queries.

Decision hardness transfers to optimization



If optimization were easy, the decision version would also be easy.

Therefore, an NP-hard decision version makes the optimization problem NP-hard too.

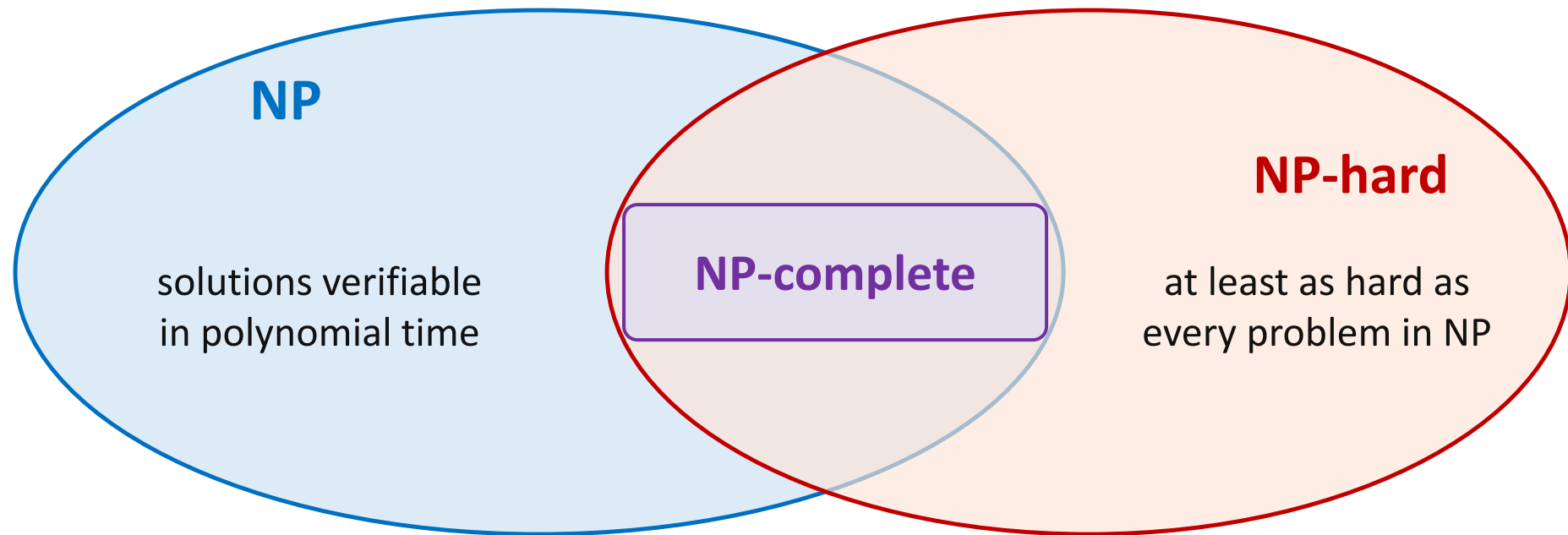
A reduction lets one solver handle another problem



x is YES for A $\Leftrightarrow$ $f(x)$ is YES for B

Notation: $A \leq_p B$ means a solver for B can solve A after this transformation.

NP-complete problems sit at the boundary of verification and hardness



NP-complete = in NP and NP-hard.

An NP-completeness proof has two obligations

1

Show $X \in \text{NP}$

Give a polynomial-time verifier
for a proposed certificate.

2

Show X is NP-hard

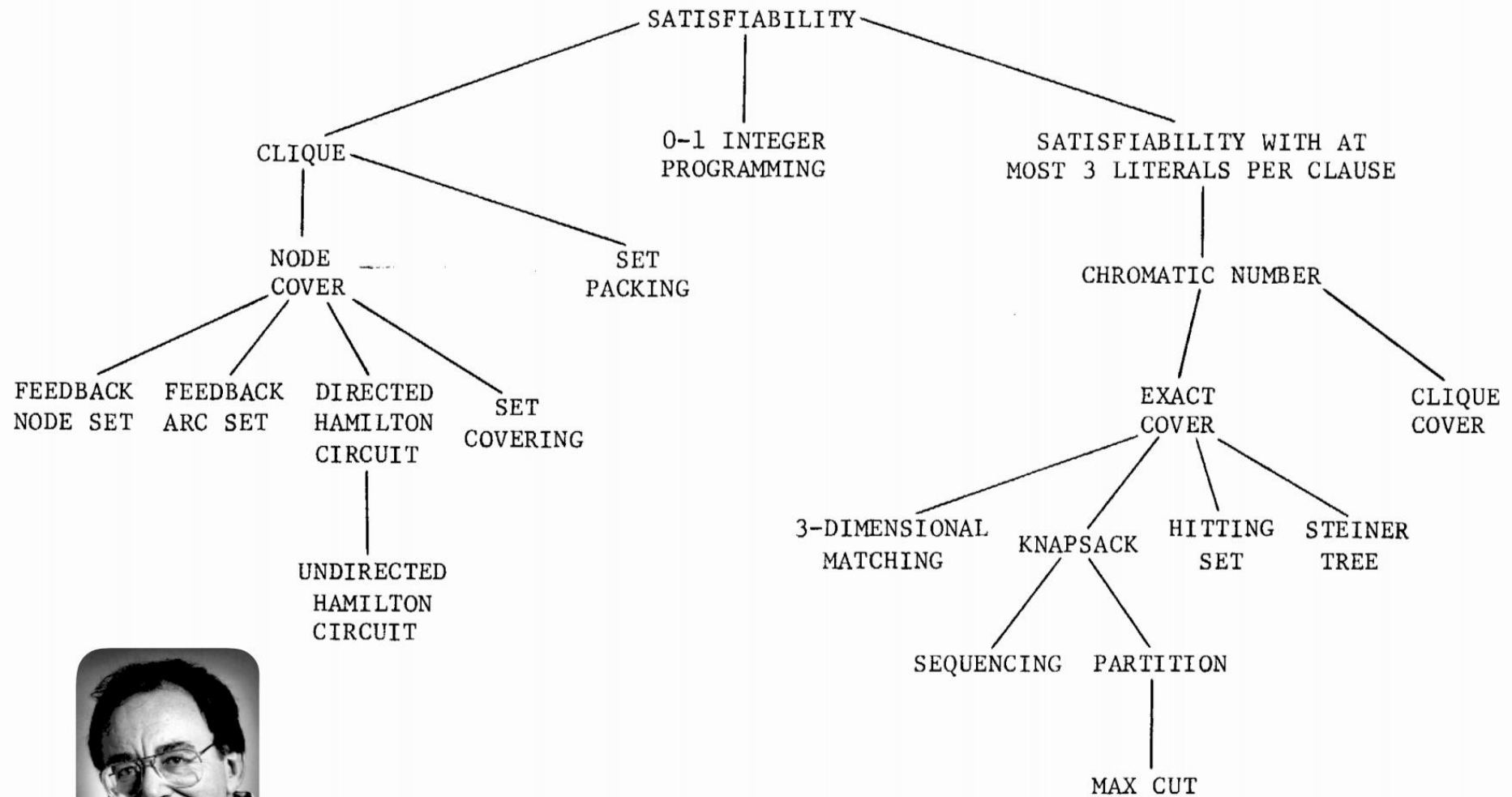
Choose a known NP-complete A
and prove $A \leq_p X$.

Membership and hardness are separate claims—prove both.

For hardness, the reduction arrow must point toward the new problem



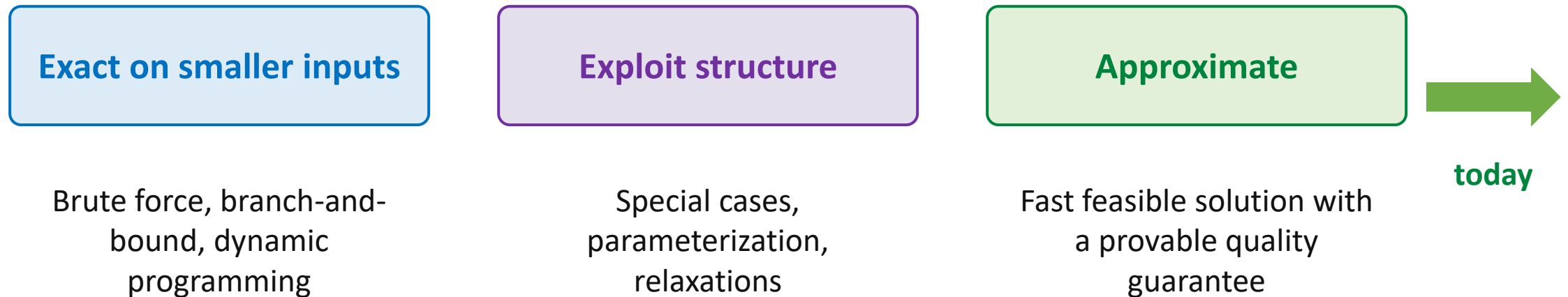
A reduction shows the target is at least as hard as the source.



Dick Karp (1972)
1985 Turing Award

FIGURE 1 - Complete Problems

When exact optimization is hard, choose the right compromise



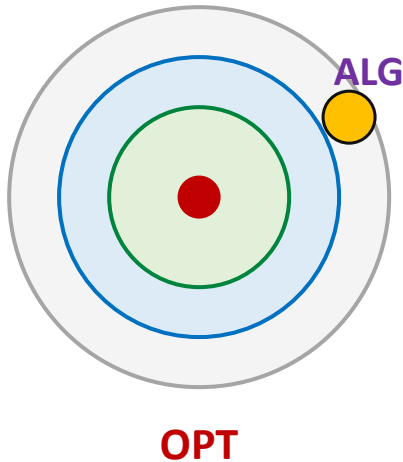
Approximation is useful only when the loss is quantified.

NP-Hard Optimization Problems



Approximation trades a little quality for speed

feasible solutions



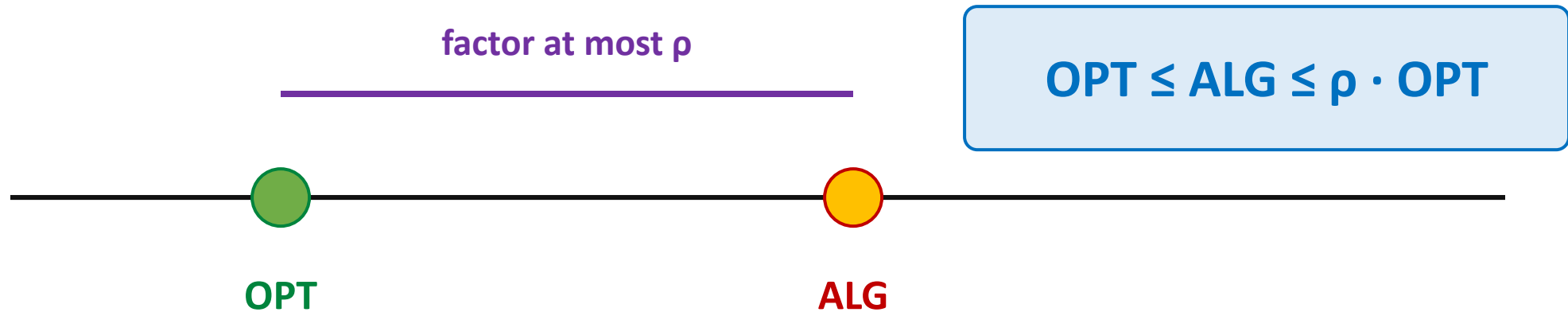
Polynomial time

+

Worst-case quality bound

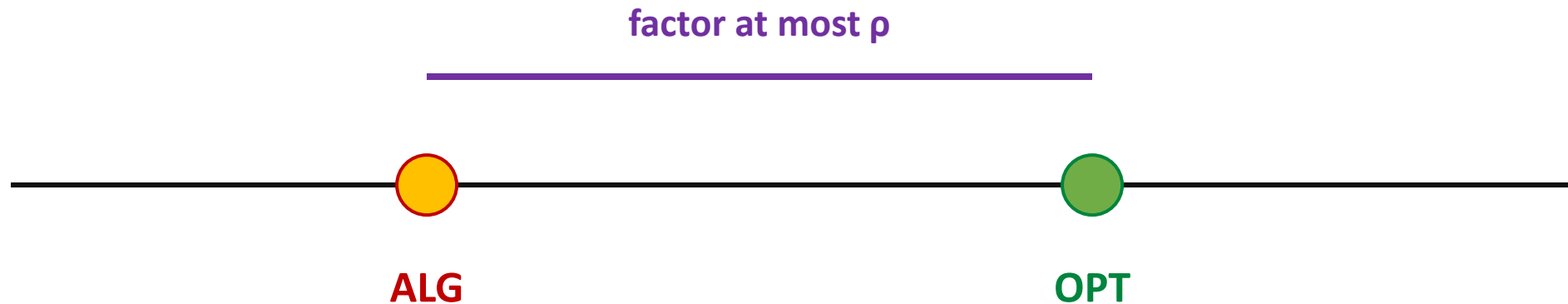
A good approximation algorithm guarantees both feasibility and "closeness" to OPT.

For minimization, ALG may be above OPT—but not by more than $\rho \geq 1$



Examples: shortest tour, set cover, scheduling cost.

For maximization, ALG may be below OPT—but not by more than $\rho \geq 1$



$$\text{OPT} / \rho \leq \text{ALG} \leq \text{OPT}$$

Examples: knapsack value, maximum cut, maximum coverage.

One ratio convention handles both minimization and maximization

minimization
 $\text{ALG} / \text{OPT} \leq \rho$



same idea

maximization
 $\text{OPT} / \text{ALG} \leq \rho$

$\max \{ \text{ALG}/\text{OPT}, \text{OPT}/\text{ALG} \} \leq \rho$

Always state which convention you use; here $\rho \geq 1$ is the approximation factor.

Common approaches



Other Approaches



All Come Together



A guarantee comes from comparing ALG and OPT on an arbitrary instance

- 1 Fix any input instance I
- 2 Name the algorithm value $ALG(I)$ and optimum $OPT(I)$
- 3 Use structure or a relaxation to relate the two values
- 4 Conclude the same inequality for every I

Knapsack: maximize value without exceeding capacity

capacity $W = 10$

selected weight ≤ 10

A
weight 4 value 10

B
weight 6 value 12

C
weight 5 value 9

D
weight 3 value 6

Best: A + B
weight 10, value 22

The combinatorial difficulty is deciding which items work well together.

Knapsack is a constrained subset-selection problem

Input items $i = 1, \dots, n$ with weight w_i and value v_i ; capacity W

Choose

a subset $S \subseteq \{1, \dots, n\}$

maximize

$$\sum_{i \in S} v_i$$

subject to

$$\sum_{i \in S} w_i \leq W$$

Integral knapsack: every item is either taken or not taken.

Fractional knapsack is solved by sorting value density



4.0
whole

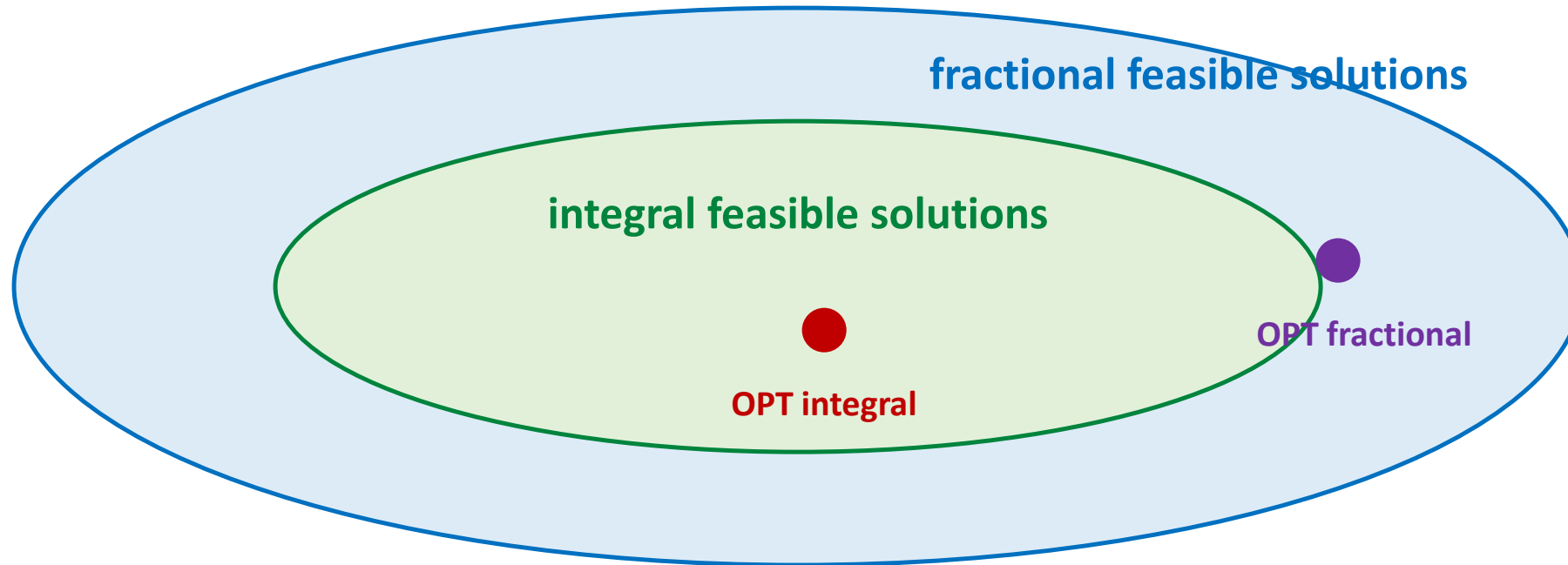
3.0
whole

2.2
fraction

1.4
stop

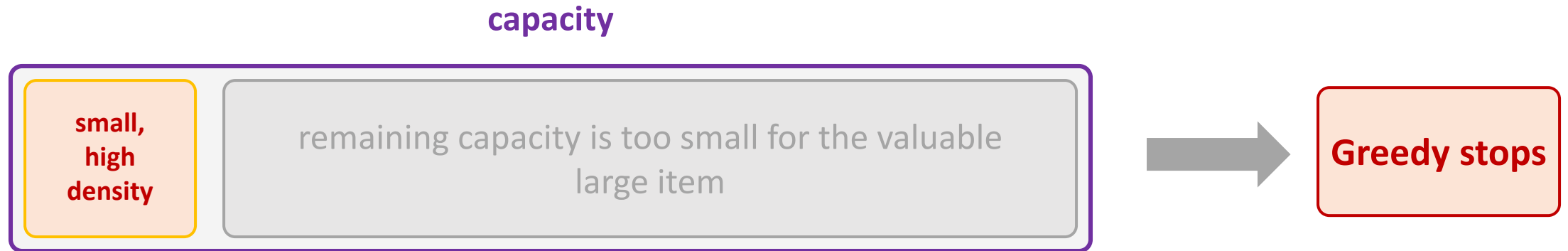
Only the first item that does not fully fit is taken fractionally.

The fractional optimum is an upper bound on integral knapsack



$OPT \leq F$, where F is the value of the fractional greedy solution.

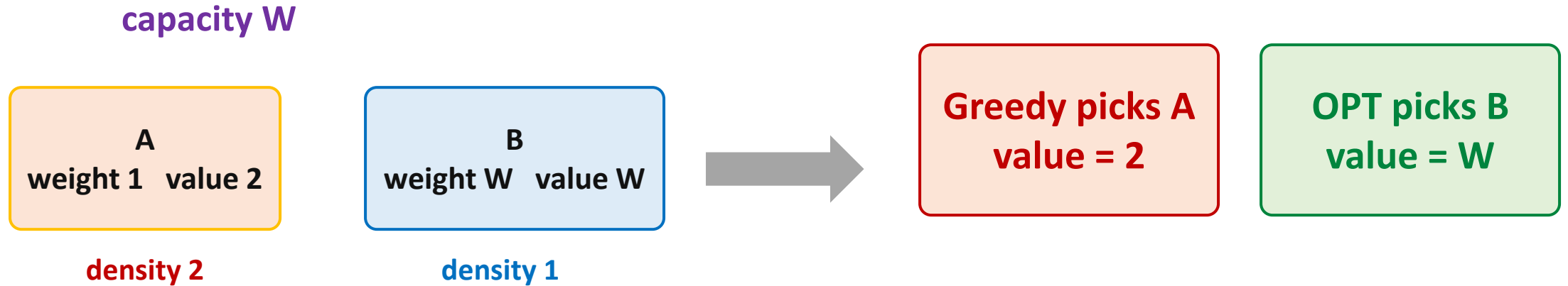
Density greedy can make a locally good choice that blocks the best item



The fractional algorithm could take a fraction; the integral algorithm cannot.

This gap is exactly what the repair must handle.

A two-item instance makes naive greedy arbitrarily bad



$$\text{ALG} / \text{OPT} = 2 / W \rightarrow 0$$

No constant approximation factor is possible for naive density greedy.

Modified greedy keeps the better of two candidates

Candidate 1
density-greedy set G

Candidate 2
best feasible single item

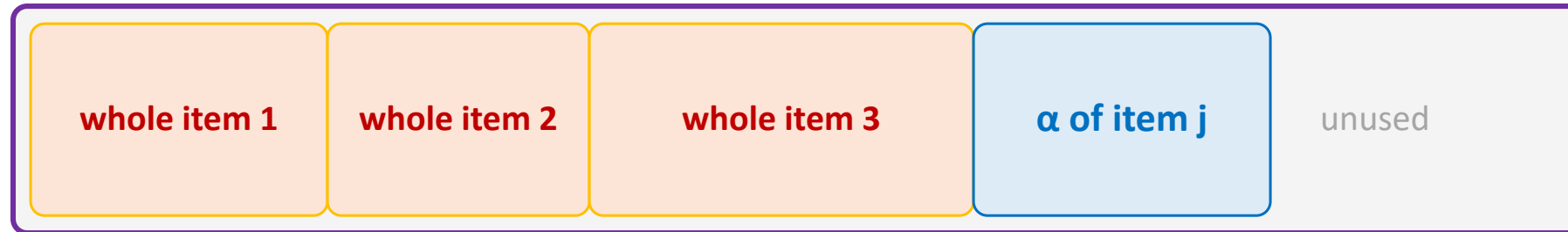
take the maximum value

$$M = \max \{ G, v_{\max} \}$$

The extra candidate protects us from the one large item greedy may block.

The fractional solution is a greedy prefix plus one fraction

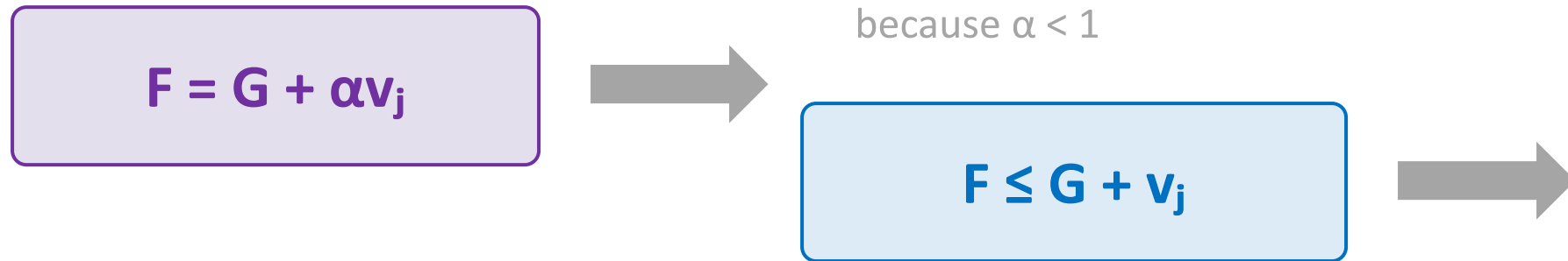
capacity W



$$F = G + \alpha v_j$$

Here G is the value of the whole-item greedy prefix and $0 \leq \alpha < 1$.

The fractional value is at most greedy plus one whole item



The best feasible single item is worth at least v_j .

$$F \leq G + v_{\max}$$

This is the key inequality; everything else is algebra.

The better candidate captures at least half of their sum



$$M \geq (G + v_{\max})/2 \geq F/2 \geq \text{OPT}/2$$

The fractional relaxation connects the algorithm to the unknown optimum.

Modified greedy is a 2-approximation for knapsack

Algorithm

Return the better of density greedy and the best feasible single item.

Guarantee

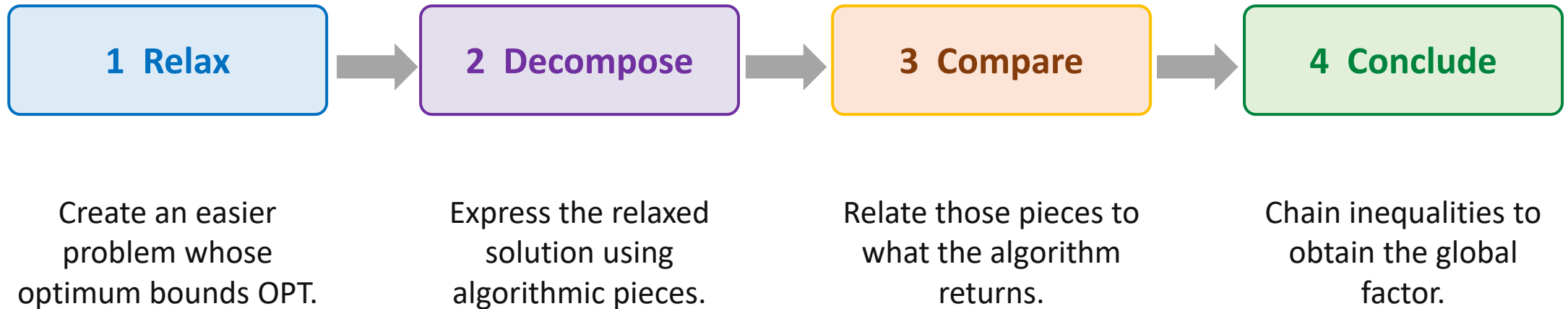
$$\text{value}(M) \geq 1/2 \cdot \text{OPT}$$

Time

Sort by density, scan once, and track the best feasible item.

A tiny algorithmic repair converts an unbounded loss into a constant guarantee.

The knapsack proof illustrates a reusable approximation template



Set cover will use a different comparison object—element prices—but the logic is the same.

An FPTAS lets us request any fixed accuracy



running time polynomial in n and $1/\epsilon$

Fully polynomial means efficient in both input size and requested precision.

Smaller ϵ improves accuracy but usually increases running time

$\epsilon = 0.20$

$\geq 80\%$ of OPT



more
work

$\epsilon = 0.10$

$\geq 90\%$ of OPT



more
work

$\epsilon = 0.01$

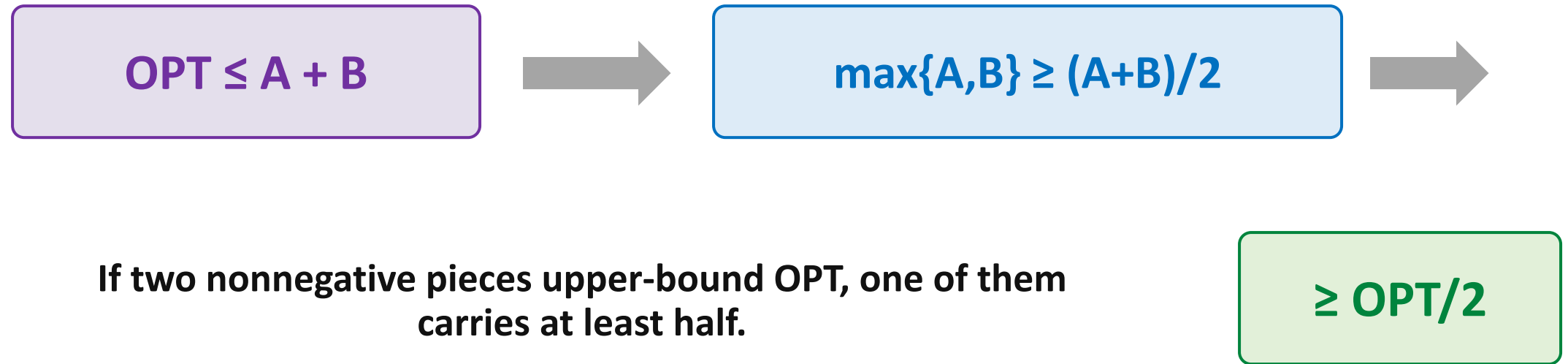
$\geq 99\%$ of OPT



more
work

An approximation scheme exposes an explicit accuracy–time tradeoff.

Returning the better of two candidates gives a generic half bound



This max-of-candidates trick appears in many simple approximation algorithms.

Quick check: maximizing the better of two objectives

Objective: $h(x) = f(x) + g(x)$, with $f, g \geq 0$



What approximation factor is guaranteed?

Hint: compare $f(x^*)$ with $f(x_f)$, and $g(x^*)$ with $g(x_g)$.

Answer: the better candidate is a 2-approximation

$$\text{OPT} = f(x^*) + g(x^*)$$

$$\leq f(x_f) + g(x_g)$$

$$\leq h(x_f) + h(x_g)$$

$$\leq 2 \cdot \max\{h(x_f), h(x_g)\}$$

$$\text{returned value} \geq \text{OPT}/2$$